Jungle Game Tech Lab Week2

[엔진 에디터]

6팀

팀원: 김기홍, 김기훈, 남윤지, 박준혁

프로젝트 개요

구현 항목	설명
UObject 코어 시스템	최상위 베이스 클래스, UUID 관리, IsA() 런타임 타입 체크, 팩토리 패턴
카메라 공간 변환 및 조작	뷰/투영 행렬, 원근·직교 전환, WASD·마우스 입력
오브젝트 피킹 및 기즈모	레이캐스팅, 삼각형 충돌 검사, 선/평면 투영 기반 이동
씬 저장 및 로드	JSON 직렬화·역직렬화, UUID 복원
콘솔 윈도우	ImGui 기반 로그 출력, 필터링, 명령어 입력
Stat 창	new/delete 오버로딩 기반 메모리·오브젝트 수 실시간 추적
윈도우 리사이즈	버퍼 재생성, AspectRatio 갱신

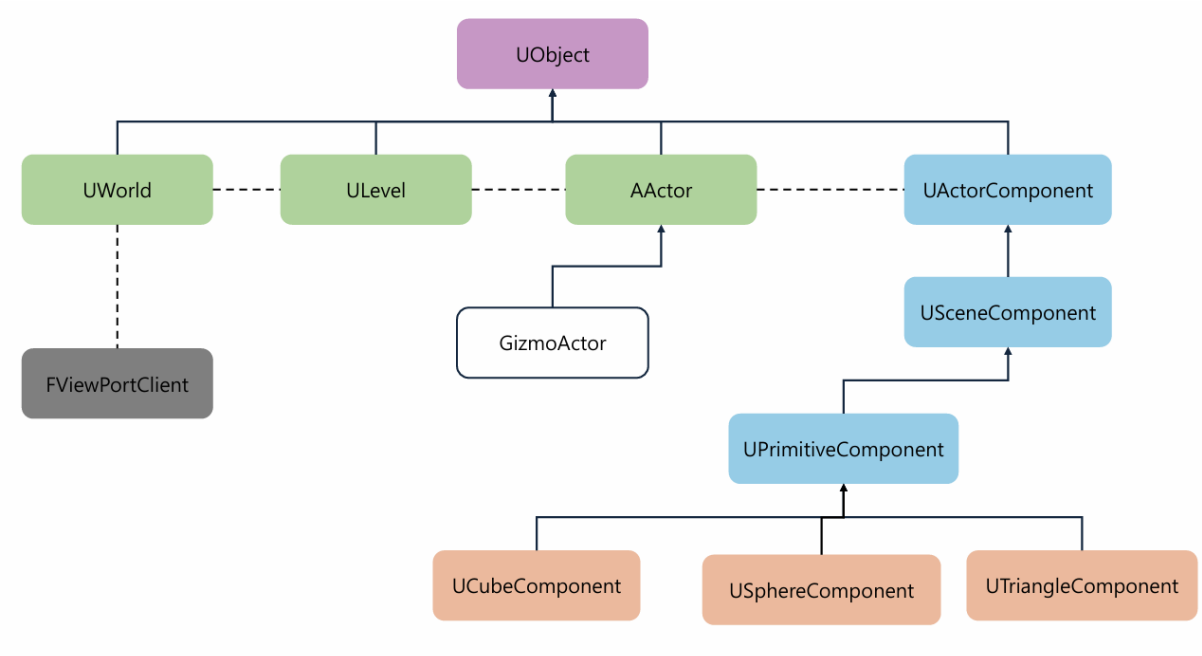
전체 구조

1. 메인루프 구조

매 프레임마다 아래 순서로 실행됩니다.

순서	시스템	담당 클래스	역할
1	입력	UInput	키보드·마우스 상태 갱신
2	뷰포트	FEditorViewportClient	카메라 이동 및 회전 처리
3	게임 로직	UWorld	액터 Tick, 피킹, 기즈모 드래그
4	렌더	Renderer	씬 렌더링
5	에디터 UI	UImGuiDrawer	ImGui 패널 갱신

2. 클래스 상속 구조



UObject 시스템

1. UUID관리

엔진 안에서 생성되는 모든 오브젝트들은 고유한 UUID를 부여받습니다. 앱을 껐다가 켜도 UUID가 유지될 수 있도록 씬 로드 시에 UUID를 새로 부여하지 않고 저장된 ID로 복원합니다. 또한 마지막으로 사용된 NextUUID 값은 JSON에 별도로 저장하여 다음 실행시에 이어서 발급하게 됩니다.

2. IsA()

클래스 체인을 순회하여 상속 관계를 런타임에 확인할 수 있는 RTTI 시스템입니다.

3. GUObjectArray

생성된 모든 UObject는 전역 TArray인 GUObjectArray에 등록되어 일괄 처리됩니다.

팩토리 패턴을 통해 NewObject<T>()로 오브젝트를 생성할 수 있도록 하고, 생성 시 UUID가 자동으로 부여되며 배열에 등록됩니다.

4. AActor

씬에 배치되는 오브젝트의 기본 단위입니다. Actor는 가지고 있는 모든 컴포넌트들을 업데이트 하고 RootComponent를 기준으로 위치와 스케일을 동기화 합니다.

카메라 시스템

1. 공간 변환

카메라는 두 가지 행렬을 조합하여 3D월드를 2D화면에 투영합니다.

행렬	역할	관련 함수
뷰 행렬 (View Matrix)	카메라 위치·방향 기준으로 월드를 카메라 공간으로 변환	GetViewMatrix()
투영 행렬 (Projection Matrix)	3D 공간을 2D 화면에 투영 (원근 또는 직교)	GetProjectionMatrix()

2. 원근 직교 전환

전환 방향	계산식	목적
직교 → 원근	D = S / (2 × tan(FOV/2))	같은 크기로 보이도록 카메라 거리 계산
원근 → 직교	S = 2 × D × tan(FOV/2)	현재 거리의 가시 범위를 직교 크기로 환산

3. 카메라 조작

입력	동작	구현 함수
W/A/S/D	카메라 전후좌우 이동	HandleKeyboardMovement()
마우스 우클릭 드래그	카메라 회전 (Pitch / Yaw)	HandleMouseRightDrag()

입력	동작	구현 함수
Alt + 마우스 우클릭 드래그	피벗 기준 오빗(Orbit) 회전	HandleMouseRightDrag()
마우스 휠	줌 인/아웃	HandleMouseWheel()
마우스 중클릭 드래그	화면 패닝	HandleMiddleMouseDrag()

오브젝트 피킹 및 기즈모

1. 2D에서 3D로 ray 변환

- 마우스 픽셀 좌표 → NDC(Normalized Device Coordinates)로 변환
- 뷰-프로젝션 행렬의 역행렬을 이용해 NDC → 월드 공간으로 역변환
- Near 평면의 점과 Far 평면의 점을 이어 레이 방향 벡터 계산

2. 충돌검사

오브젝트의 메시를 구성하는 버텍스를 3개씩 묶어 삼각형 단위로 레이와의 교차를 검사합니다. 오브젝트의 변환(위치·회전·스케일)이 반영되도록 레이를 오브젝트의 로컬 좌표계로 변환한 후 계산합니다.

3. 기즈모

조작	방식	원리
X / Y / Z 축 화살표	선 투영 (Line Projection)	마우스 레이와 해당 축 사이의 최근접점을 구해 축 방향으로만 이동 제한
Center (중앙점)	평면 투영 (Plane Projection)	카메라 방향을 법선으로 하는 가상 평면에 레이를 교차시켜 자유 이동

4. 속성창 연동

ImGui 속성창(Property Window)에서 선택된 오브젝트의 위치·회전·스케일을 수치로 직접 편집할 수 있습니다. 기즈모 드래그와 속성창 입력은 동일한 RootComponent 변환 데이터를 공유합니다.

씬 저장 및 로드

1. 개요

USceneSerializer는 현재 씬의 상태를 JSON 파일로 저장하고, 이를 다시 불러와 씬을 복원합니다. nlohmann/json 라이브러리를 사용하며, 파일 확장자는 ".Scene"입니다. ImGui 패널의 Save/Load 버튼과 연결되어 동작합니다.

2. 저장 데이터 구조

```
{
  "Version": 1,
  "NextUUID": 5,
  "Primitives": {
    "1": {
        "Location": [0.0, 0.0, 3.0],
        "Rotation": [0,0,0],
        "Scale": [0.5,0.5,0.5],
        "Type": "Cube"
        }
  }
}
```

3. 저장 흐름

- 씬의 모든 액터를 순회하여 PrimitiveComponent 가 있는 액터만 수집
- 각 액터의 UUID, 위치·회전·스케일·타입을 JSON 으로 직렬화
- {저장 경로}/{씬 이름}.Scene 파일로 출력

4. 로드 흐름

- Scene 파일을 읽어 JSON 파싱
- 현재 씬 초기화 (ClearScene) 및 NextUUID 복원
- 각 항목을 FSpawnParameters 로 변환하여 SpawnActorFromEditor 호출
- 저장 당시의 UUID 포함 액터 완전 복원

콘솔 윈도우

ImGui 데모(imgui_demo.cpp)의 AppConsole 구조체를 기반으로 엔진 전용 콘솔 윈도우를 구현하였습니다. UE_LOG 매크로를 활용하여 코드 어디서든 로그를 출력할 수 있습니다.

Stat 창

new/delete 연산자 오버로딩을 통해 엔진 전체의 메모리 할당을 추적합니다. FMemory 클래스에서 TotalAllocationBytes와 TotalAllocationCount를 관리하며, ImGui Stat 창에서 실시간으로 표시합니다.

윈도우 리사이즈

윈도우 크기가 변경될 때 버퍼와 뷰포트를 재생성하여 화면이 정상적으로 유지되도록 합니다. WM_SIZE 메시지를 감지하면 Renderer::OnResize()가 호출됩니다.

- WM_SIZE 감지 → bResized 플래그 설정
- 다음 프레임 시작 시 Renderer::OnResize(width, height) 호출
- 기존 RenderTargetView 해제
- SwapChain::ResizeBuffers()로 버퍼 크기 재조정
- RenderTargetView 재생성
- 뷰포트 및 AspectRatio 갱신 → 카메라 투영 행렬 자동 반영
- ImGui DisplaySize 갱신